

Reviewer Pack — Minimal Rank of Quandle Representations Bounds XOR Circuit W...

Entry 97a360972c8e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 15:31:01 UTC

Minimal Rank of Quandle Representations Bounds XOR Circuit Weights

Entry ID: 97a360972c8e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 15:31:01 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: Boolean Function Complexity

Statement:

{‘quantitative_relation’: ‘For all XOR circuits C with n gates, the minimal rank $\rho(C)$ of the quandle representation of C satisfies $\rho(C) = \Theta(n^{1/2})$ ’, ‘falsifier_friendly_formulation’: ‘If there exists an XOR circuit C with n gates such that the minimal rank $\rho(C) < n^{1/2}$, then the conjecture is false.’}

Rationale (proposer’s reasoning):

{‘exposure_of_structure’: ‘Quandle theory provides a framework for studying algebraic structures related to symmetry and regularity. A connection between quandle representations and circuit complexity could expose new structures in XOR circuits, potentially leading to improved lower bounds on circuit size.’, ‘influential_work’: ‘While quandles have not been widely applied to complexity theory, the study of their representations is well-developed. This offers a promising avenue for exploration.’}

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a6d0b8aa035bcb1c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the average minimal rank $\rho(C)$ of quandle representations for XOR circuits with n gates is within $\pm 30\%$ of $n^{1/2}$, over at least 30 seeds, and the standard deviation of $\rho(C)$ does not exceed 10% of $n^{1/2}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.8s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        factor = A[i][i]
        if factor == 0:
            continue
        for j in range(n):
            A[i][j] /= factor
        for j in range(m):
            if i != j:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
    rank = sum(1 for row in A if any(row))
    return rank

def compute_minimal_rank(circuit):
    n = len(circuit)
    Q = [[0] * (n + 1) for _ in range(n + 1)]
    for i, gate in enumerate(circuit):
        x, y = gate
        Q[x][y] += 1
        Q[y][x] -= 1
    rank = gaussian_elimination(Q)
```

```

    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([10, 20, 30, 40])
    instances_tested = 30
    total_rank = 0

    for _ in range(instances_tested):
        circuit = []
        for _ in range(n):
            x = random.randint(0, n-1)
            y = random.randint(0, n-1)
            while x == y:
                y = random.randint(0, n-1)
            circuit.append((x, y))

        rank = compute_minimal_rank(circuit)
        total_rank += rank

    avg_rank = total_rank / instances_tested
    conjecture_holds = abs(avg_rank - math.sqrt(n)) <= 0.3 * math.sqrt(n) and (math.sqrt(n) - 10 < 0)
    counterexample = "" if conjecture_holds else f"avg_rank={avg_rank}, sqrt(n)={math.sqrt(n)}"

    return {
        "metric_name": "minimal_rank",
        "metric_value": avg_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    avg_rank = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - avg_rank) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_rank} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {"seed": 11, ...run_trial output...}
TRIAL: {"seed": 23, ...run_trial output...}
TRIAL: {"seed": 37, ...run_trial output...}
TRIAL: {"seed": 53, ...run_trial output...}
TRIAL: {"seed": 71, ...run_trial output...}
TRIAL: {"seed": 89, ...run_trial output...}
TRIAL: {"seed": 103, ...run_trial output...}
TRIAL: {"seed": 127, ...run_trial output...}
TRIAL: {"seed": 149, ...run_trial output...}
TRIAL: {"seed": 167, ...run_trial output...}
TRIAL: {"seed": 191, ...run_trial output...}
TRIAL: {"seed": 211, ...run_trial output...}
TRIAL: {"seed": 233, ...run_trial output...}
TRIAL: {"seed": 257, ...run_trial output...}
TRIAL: {"seed": 277, ...run_trial output...}
TRIAL: {"seed": 311, ...run_trial output...}
TRIAL: {"seed": 347, ...run_trial output...}
TRIAL: {"seed": 389, ...run_trial output...}
TRIAL: {"seed": 421, ...run_trial output...}
TRIAL: {"seed": 463, ...run_trial output...}
TRIAL: {"seed": 503, ...run_trial output...}
TRIAL: {"seed": 547, ...run_trial output...}
TRIAL: {"seed": 593, ...run_trial output...}
TRIAL: {"seed": 631, ...run_trial output...}
TRIAL: {"seed": 677, ...run_trial output...}
TRIAL: {"seed": 727, ...run_trial output...}
TRIAL: {"seed": 773, ...run_trial output...}
TRIAL: {"seed": 821, ...run_trial output...}
TRIAL: {"seed": 877, ...run_trial output...}
TRIAL: {"seed": 929, ...run_trial output...}
```

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ea98ec0.py", line 96, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ea98ec0.py", line 96, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the support conditions for the conjecture. | next: Investigate and fix the crash in the test code to continue testing the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17597
2	propose	ollama_remote	glm4:latest	0	0	10482
3	preregistration	ollama_remote	glm4:latest	0	0	6425
4	novelty	ollama_remote	glm4:latest	0	0	4851
5	novelty	ollama_remote	glm4:latest	0	0	5320
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43052
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13120
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12237
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10830
10	judge	ollama_remote	glm4:latest	0	0	10881

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 134795 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/97a360972c8e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/97a360972c8e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/97a360972c8e.tar.gz (if generated)